

PLAN · RAG + MCP FOR QA KNOWLEDGE

QA Knowledge Base for LLM Question Answering

How to store test cases, Jira tickets and source code in MongoDB as **Markdown + vectors + links**, and how an LLM uses MCP tools to answer simple and complex questions with real sources.

MongoDB Atlas Vector Search

Markdown chunks

Hybrid search (\$rankFusion)

Graph links (\$graphLookup)

MCP tools

Jira + Source code

1 · The big picture

2 · Three ways to find things

3 · The JSON format (5 types)

4 · How everything links

5 · Building the data (pipeline)

6 · Indexes in MongoDB

7 · MCP tools design

8 · How the LLM answers

9 · Demos: 5 question types

10 · Why this helps the LLM explain

11 · Step-by-step rollout

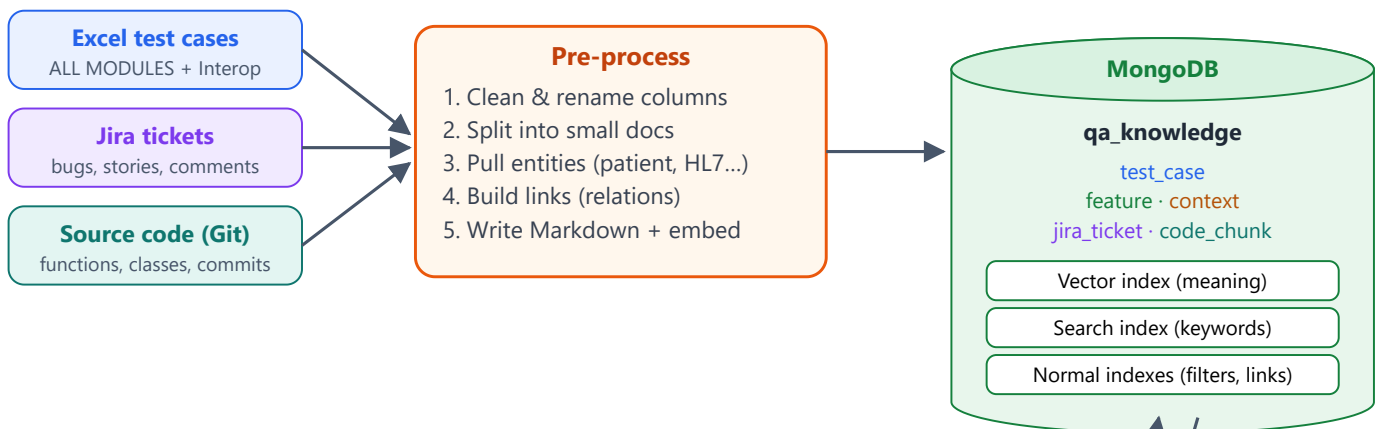
12 · Notes & limits

Source data: ALL MODULES_TCs.xlsx (~2,770 test cases, Firebird + MySQL tracks) ·
InteropsTeamFunctionalityDetails.xlsx (19 interop sheets) · Prepared 2 Oct 2026

1 The big picture

Data comes in on the top row. Questions go out on the bottom row. MongoDB sits in the middle and holds everything in **one collection** called `qa_knowledge`.

① INGEST (once, then on every change)



② ASK (every question)

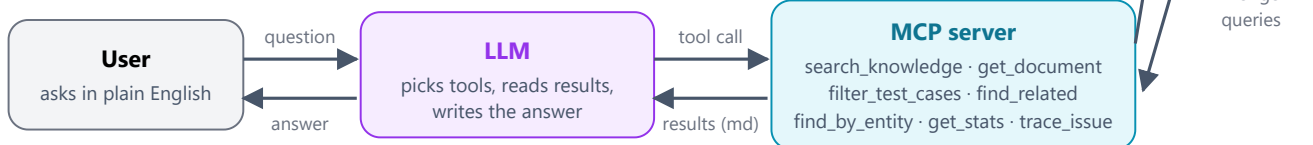


Figure 1 — Ingest on top, ask on the bottom. The LLM never touches MongoDB directly; it only uses MCP tools.

The one idea to remember

Markdown = what the LLM reads

Clean text with title, steps and expected result. The LLM can quote it word for word.

Vector = how we find by meaning

A list of numbers made from the Markdown. Similar meaning gives similar numbers.

Fields + links = exact facts

Status, owner, module and `relations[]` give exact filters and connections.

All three live in the **same JSON document**. One search can filter by fields, rank by meaning, and then follow links.

2 Three ways to find things

Users ask different kinds of questions. Each kind needs a different search. The LLM picks the right one, or combines them.

Keyword search

Finds **exact words**: codes, IDs and names.

"Show test INT-RxFill-Naproxen-2a"

"Tests for ADT-A08"

MongoDB: `$search` (Atlas Search)

Semantic search

Finds **same meaning**, even when the words are different.

"What if the pharmacy can't give the full quantity?"

→ finds the "Partially Dispensed / Out of Stock" test

MongoDB: `$vectorSearch`

Relationship search

Follows **links** between docs: test → bug → code.

"This test failed. Which Jira bug and which code file are related?"

MongoDB: `$graphLookup`

Plus two helpers

Hybrid = keyword + semantic together

Both searches run, and their result lists are merged with `$rankFusion` (MongoDB 8.1+). This is best when a question has a code *and* an idea: "ADT-A08 insurance update test in env 93".

Stats = counting

Plain `aggregate()` with `$group`. No vectors needed.
"How many MySQL tests failed per module?"

Which search for which question?

If the question...	Use	Example
has an ID, code, name or exact phrase	Keyword	"Testcase - 2 RxChange steps"
describes a situation in normal words	Semantic	"How do we test a prescription being cancelled?"
has both a code and an idea	Hybrid	"VXU message when vaccine is given"
asks "why", "what's connected" or "what's impacted"	Relationship	"What breaks if CancelRx sending fails?"
asks "how many", "which most" or "% passed"	Stats	"Pass rate of Billing on Firebird"
filters by status, owner, module or env	Exact filter	"Marcelo's failed tests"

3 The JSON format (5 document types)

There is one collection with a `doc_type` field. Every type shares the same "common part", so one search covers everything.

test_case

one Excel test row

feature

eRx, ADT, eFax...

context

test data, config,
preconditions

jira_ticket

one bug or story

code_chunk

one function or class

The common part (every document has these)

Field	What it is for
<code>_id</code>	A stable, readable ID such as <code>TC:INTEROP:ERx:R27</code> . Links point to it.
<code>doc_type</code>	Says which of the 5 types this is, so searches can filter by type.
<code>source</code>	Where it came from: sheet and row, Jira key, or file path. Used for citations .
<code>hierarchy.breadcrumb</code>	Where it lives, e.g. <i>Interop > eRx > Testcase - 1 > CancelRx</i> .
<code>entities[]</code>	Typed "things" mentioned (patient, drug, HL7 message, file). These join docs across sources.
<code>relations[]</code>	Arrows to other docs: BELONGS_TO, DEPENDS_ON, AFFECTS, FIXED_BY...
<code>search.keywords</code>	Exact words for keyword search, including spelling variants found in the sheets.
<code>content_md</code>	Clean Markdown. This is what the LLM reads and what gets embedded.
<code>embedding</code>	The vector plus a hash of the text, so we re-embed only when the text changes.

TEST_CASE Real example: eRx sheet, row 27 (it failed)

```
{
  "_id": "TC:INTEROP:ERx:R27",
  "doc_type": "test_case",
  "schema_version": "1.0",

  "source": {                                     // where it came from (for citations)
    "workbook": "InteropsTeamFunctionalityDetails",
    "sheet": "eRx",
    "row": 27,
    "raw_headers": { "title": "Test Scenario", "steps": "Steps", "expected": "Excepted Result" },
    "content_hash": "sha256:9f2c...",
    "ingested_at": "2026-10-02T10:30:00Z"
  },

  "hierarchy": {                                  // where it Lives
    "domain": "Interop",
    "module": "eRx",
    "section": "CancelRx",
    "group_id": "Testcase - 1",
    "sequence_no": 26,
    "breadcrumb": "Interop > eRx > Testcase - 1 > CancelRx"
  },

  "title": "Validate - RxCancelResponse to OP",
  "preconditions": [],
  "steps": [                                       // array, so the LLM can say "step 3"
    "In OP > e-prescribe > Cancelled/Denied cancel tab, double click on the CancelResponse transaction.",
    "Or click on the message icon of that transaction.",
    "It opens the XML.",
  ]
}
```

```

    "Validate all the fields of CancelRx request are populated correctly: Denial Reason, Code & Type."
  ],
  "expected_result": "All the fields must be populated correctly in the XML of the CancelResponse.",

  "context_ref": "CTX:INTEROP:ERx:TESTCASE-1", // full test data stored once
  "context_snapshot": {                       // small copy so the doc stands alone
    "patient_id": "118",
    "medication": "Augmented Betamethasone 0.05% Topical Ointment",
    "pharmacy": "Shollenberger Pharmacy",
    "substance_class": "NON_CONTROLLED"
  },

  "entities": [                               // typed things, used to join docs
    { "type": "patient",      "value": "118" },
    { "type": "medication",   "value": "Augmented Betamethasone 0.05% Topical Ointment" },
    { "type": "message_type", "value": "CancelRxResponse", "standard": "NCPDP SCRIPT" },
    { "type": "system",      "value": "OP" },
    { "type": "system",      "value": "Surescripts" }
  ],
  "environments": ["ID1", "ID93", "ID106"],

  "runs": [                                   // every result, one per track / cycle
    { "track": "default", "owner": null, "status": "FAIL", "status_raw": "Fail",
      "comment": null, "executed_at": null, "recorded_at": "2026-10-02T10:30:00Z" }
  ],

  "relations": [                               // arrows to other documents
    { "type": "BELONGS_TO",   "target": "FEAT:INTEROP:ERx" },
    { "type": "USES_CONTEXT", "target": "CTX:INTEROP:ERx:TESTCASE-1" },
    { "type": "DEPENDS_ON",   "target": "TC:INTEROP:ERx:R26", "why": "CancelResponse must be sent first" },
    { "type": "SAME_SCENARIO", "target": "TC:INTEROP:ERx:R57", "why": "same check, patient 123" }
  ],

  "quality": { "flags": ["HEADER_TYPO_FIXED"], "missing_fields": [] },

  "search": {
    "keywords": ["RxCancelResponse", "CancelRx", "XML", "Denial Reason", "Surescripts", "118"],
    "aliases": ["Cancel Response", "CanelResponse", "Cancel Rx"]
  },

  "content_md": "# eRx > Testcase - 1 > CancelRx\n## Validate - RxCancelResponse to OP\n**Context:** Patient 118, Augmented Betamethasone, Shollenberger Pharmacy\n**Depends on:** Send - RxCancelResponse to OP\n### Steps\n1. ... \n### Expected\nAll the fields must be populated correctly ...",

  "embedding": { "model": "<your-embedding-model>", "dims": 1536,
    "text_hash": "sha256:41ab...", "vector": [0.0142, -0.0418, "..."] }
}

```

Status is NOT in `content_md` or the vector. Status changes every test cycle. Keeping it only in `runs[]` means a status change never forces a re-embed.

TEST_CASE Same format, ALL MODULES row (FrontDesk row 3). Only these parts differ.

```

{
  "_id": "TC:CORE:FrontDesk:R3",
  "hierarchy": { "domain": "Core", "module": "FrontDesk",
    "section": "Login: Show a popup when the user enters the wrong password",
    "breadcrumb": "Core > FrontDesk > Login" },
  "title": "Verify warning popup shows when user logs in with wrong username and password",
  "attributes": { "tc_source": "Insprint", "is_high_level": true },
  "runs": [
    { "track": "Firebird", "scope": "HIGH_LEVEL", "owner": "Marcelo", "status": "NOT_RUN" },
    // two databases = two runs of the same test
  ]
}

```

```
    { "track": "MySQL",      "scope": "FULL_REGRESSION", "owner": "Marcelo", "status": "NOT_RUN" }  
  ],  
  "context_ref": null,  
  "environments": []  
}
```

FEATURE One per row of the *IntropsFeatures* sheet

```
{
  "_id": "FEAT:INTEROP:HUB_ADT",
  "doc_type": "feature",
  "name": "ADT",
  "parent": "FEAT:INTEROP:HUB",
  "test_sheet": "HUB_ADT&SIU",
  "environments": ["ID93", "ID8", "ID10"],
  "directions": ["INBOUND", "OUTBOUND"],
  "sub_features": [
    { "code": "ADT_A04", "action": "Patient Creation" },
    { "code": "ADT_A08", "action": "Patient Updation" },
    { "code": "SIU_S12", "action": "Appointment Creation" }
  ],
  "doc_links": [{ "title": "ADT_TestingDocForAllEnv", "url": "https://docs.google.com/..." }],
  "content_md": "# Feature: ADT (HUB)\nRuns in ID93, ID8, ID10 ...",
  "embedding": { "...": "..." }
}
```

CONTEXT Parent blocks, preconditions and config, stored once

```
{
  "_id": "CTX:INTEROP:erX:TESTCASE-1",
  "doc_type": "context",
  "context_kind": "TEST_DATA", // or PRECONDITION / CONFIGURATION
  "applies_to": { "sheet": "erX", "rows": [3, 27] },
  "data": {
    "patient_id": "118", "patient_name": "Zachary Delaplaine",
    "pharmacy": "Shollenberger Pharmacy", "primary_dx": "Psoriasis",
    "medication": "Augmented Betamethasone 0.05% Topical Ointment",
    "quantity": 14.555, "unit": "Gram", "refills": 3, "substance_class": "NON_CONTROLLED"
  },
  "content_md": "# Test data: erX Testcase - 1\n- Patient 118 ...",
  "embedding": { "...": "..." }
}
```

JIRA_TICKET One per ticket **EXAMPLE DATA**

```
{
  "_id": "JIRA:OPI-2045",
  "doc_type": "jira_ticket",
  "source": { "system": "jira", "key": "OPI-2045", "url": "https://yourcompany.atlassian.net/browse/OPI-2045",
    "updated_at": "2026-09-28T14:02:00Z" },
  "issue_type": "Bug",
  "status": "In Progress",
  "priority": "High",
  "summary": "CancelRx response XML missing Denial Reason Code",
  "components": ["erX"],
  "fix_version": "Release 2.1",
  "assignee": "dev.team.erx",
  "labels": ["surescripts", "cancelrx"],
  "entities": [
    { "type": "message_type", "value": "CancelRxResponse" },
    { "type": "patient", "value": "118" }
  ],
  "relations": [
    { "type": "BELONGS_TO", "target": "FEAT:INTEROP:erX" },
    { "type": "AFFECTS", "target": "TC:INTEROP:erX:R27", "how": "QA linked the failed test" },
    { "type": "FIXED_BY", "target": "CODE:op-erx:src/Surescripts/CancelRxResponseParser.cs#Parse",
      "how": "commit message mentions OPI-2045" }
  ],
}
```

```

"content_md": "# OPI-2045 · CancelRx response XML missing Denial Reason Code\n**Status:** In Progress ·  

**Priority:** High\n## Description\n...\n## Comments (summary)\n- QA: fails on ID93 with patient 118 ...",
"embedding": { "...": "..." }
}

```

CODE_CHUNK One per function or class EXAMPLE DATA

```

{
  "_id": "CODE:op-erx:src/Surescripts/CancelRxResponseParser.cs#Parse",
  "doc_type": "code_chunk",
  "source": { "repo": "op-erx", "path": "src/Surescripts/CancelRxResponseParser.cs",
    "lines": [42, 118], "commit": "a91f3c2", "branch": "main" },
  "language": "csharp",
  "symbol": { "kind": "method", "name": "Parse", "class": "CancelRxResponseParser" },
  "summary": "Reads a Surescripts CancelRxResponse XML and saves denial reason, code and type on the Rx.",
  "entities": [
    { "type": "message_type", "value": "CancelRxResponse" },
    { "type": "xml_field", "value": "DenialReasonCode" }
  ],
  "relations": [
    { "type": "IMPLEMENTS", "target": "FEAT:INTEROP:erx" },
    { "type": "CALLS", "target": "CODE:op-erx:src/Rx/RxRepository.cs#SaveCancelStatus" },
    { "type": "TESTED_BY", "target": "TC:INTEROP:erx:R27" }
  ],
  "content_md": "# CancelRxResponseParser.Parse\n**What it does:** ...\n```csharp\npublic CancelResult  

Parse(XDocument xml) { ... }\n```",
  "embedding": { "...": "..." }
}

```

4 How everything links

Each arrow is one entry in a document's `relations[]`. Once these arrows exist, the LLM can walk from a failed test to the bug and then to the code.

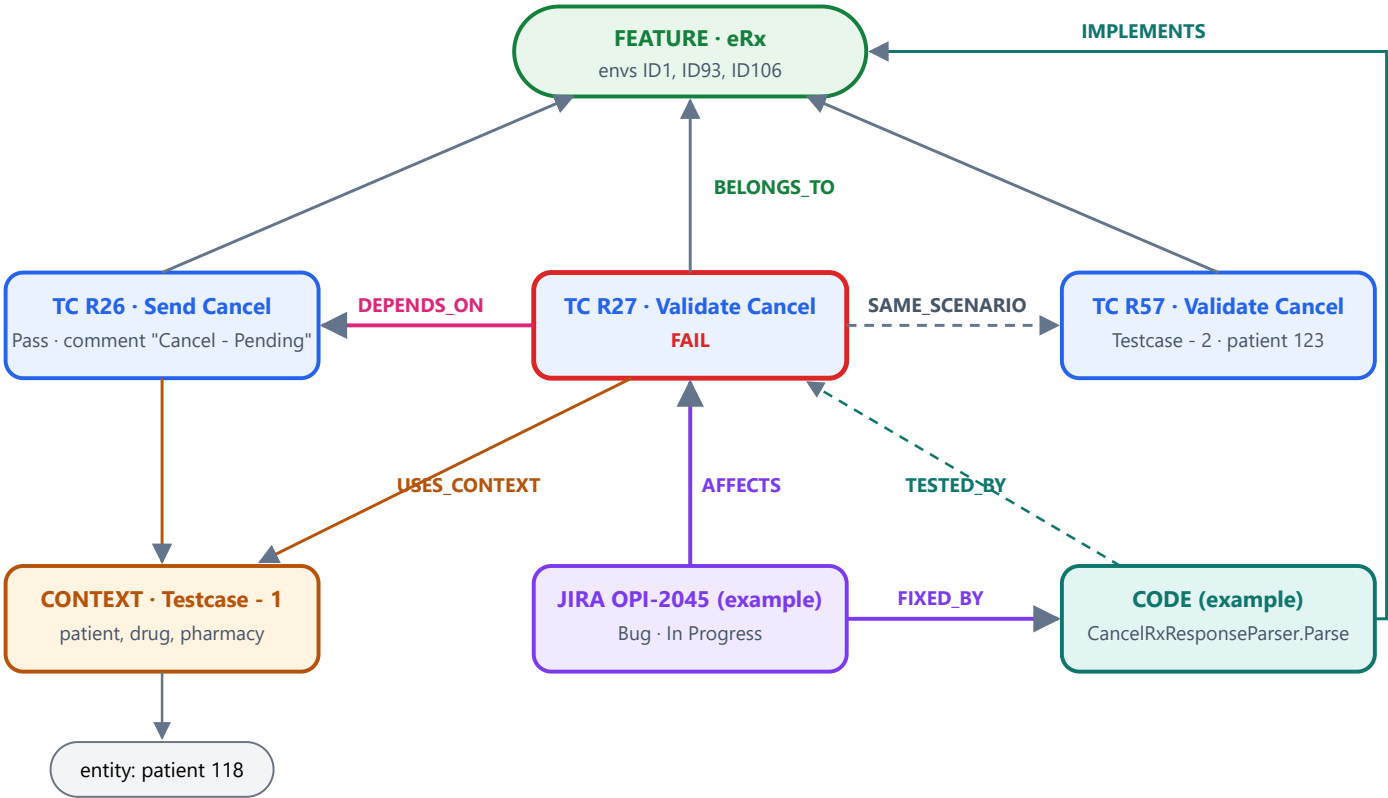


Figure 2 — A small slice of the graph. Real data: TC R26/R27/R57 and Testcase - 1. Jira and code nodes are examples.

How the links are made (automatically)

Link	Rule	Example
BELONGS_TO	Sheet name or Jira component matches a feature	sheet "eRx" → FEAT:eRx
USES_CONTEXT	Row sits under a parent block or precondition row	R27 is under "Testcase - 1"
DEPENDS_ON	Within a group, "Validate X" follows "Send X"	R27 depends on R26
SAME_SCENARIO	Same title in another group (different test data)	R27 ↔ R57
AFFECTS	Jira mentions a TC id or title, or QA links it	OPI-2045 → R27
FIXED_BY	A commit message contains the Jira key; changed functions get the link	"OPI-2045 fix denial code"
IMPLEMENTS / TESTED_BY	Code and tests share the same entity (e.g. message type CancelRxResponse)	Parser ↔ R27

Shared entities are the "glue". Patient 118 appears in eRx and ePA. CancelRxResponse appears in a test, a ticket and a code file. One query on `entities.value` finds all of them.

5 Building the data (pipeline)

One script per source. They all write the same JSON shape into the same collection.

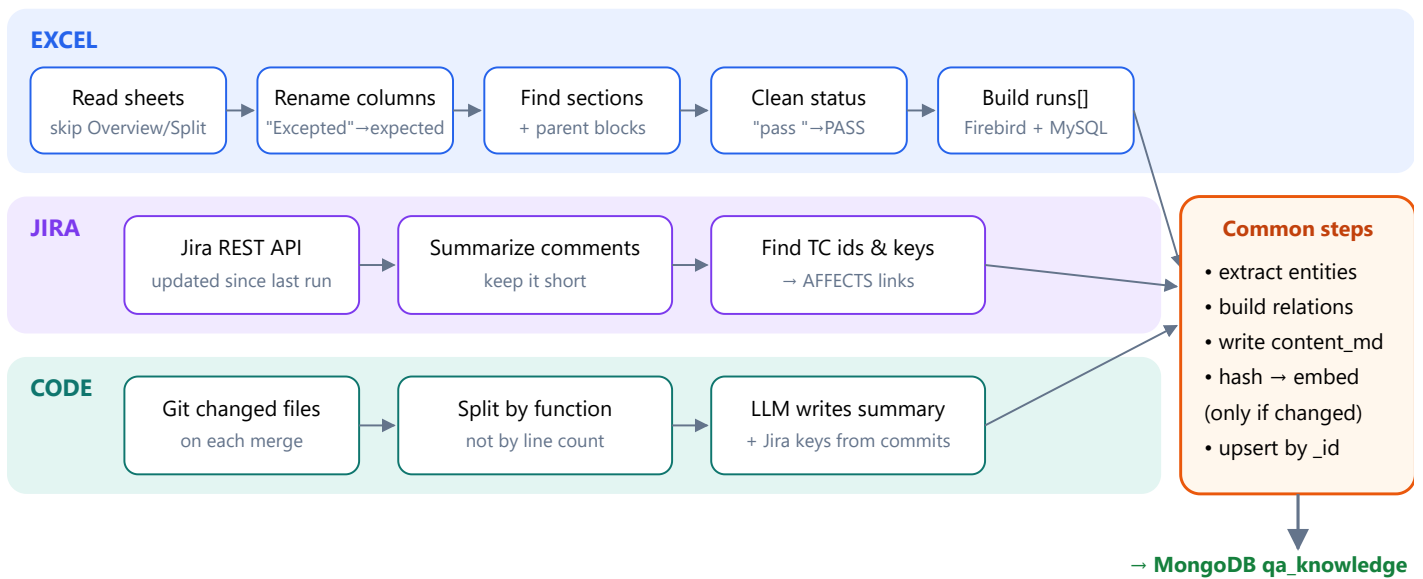


Figure 3 — Three input lanes feed the same common steps.

Storing Jira tickets: simple rules

- **One document per ticket.** Summary, description and a short comment summary go in `content_md`.
- Status, priority, assignee and fix version are **fields**, not embedded text.
- Fetch only tickets **updated since the last run** (JQL `updated >= -1d`).
- If a comment thread is very long, split it into a second chunk with the same `source.key`.

Storing source code: simple rules

- **One document per function or class.** Never cut code in the middle of a function.
- Add a **plain-English summary** written by the LLM. Questions are in English, so the summary helps semantic search a lot.
- Store `commit` and `lines` so answers can point to `file:line`.
- Re-index only files changed in each merge. Remove docs for deleted files.

6 Indexes in MongoDB

Three kinds of index on one collection. Each one powers one kind of search.

VECTOR INDEX for `$vectorSearch`

```
{
  "fields": [
    { "type": "vector", "path": "embedding.vector", "numDimensions": 1536, "similarity": "cosine" },
    { "type": "filter", "path": "doc_type" },
    { "type": "filter", "path": "hierarchy.module" },
    { "type": "filter", "path": "environments" },
    { "type": "filter", "path": "runs.status" }
  ]
}
```

SEARCH INDEX for `$search` (keywords)

```
{
  "mappings": {
    "dynamic": false,
    "fields": {
      "title": { "type": "string" },
      "search.keywords": { "type": "string", "analyzer": "lucene.keyword" },
      "search.aliases": { "type": "string" },
      "content_md": { "type": "string" },
      "entities": { "type": "document", "fields": { "value": { "type": "string" } } }
    }
  }
}
```

NORMAL INDEXES for filters, stats and links

```
db.qa_knowledge.createIndex({ doc_type: 1, "hierarchy.module": 1 })
db.qa_knowledge.createIndex({ "runs.track": 1, "runs.status": 1 })
db.qa_knowledge.createIndex({ "relations.target": 1 }) // fast $graphLookup
db.qa_knowledge.createIndex({ "entities.type": 1, "entities.value": 1 })
db.qa_knowledge.createIndex({ "source.key": 1 }) // Jira key Lookup
```

7 MCP tools design

The MCP server is a small program between the LLM and MongoDB. It offers a few **clear, read-only tools**. The LLM chooses which tool to call and with what inputs.

Tool	Inputs	Runs in MongoDB	Use it for
search_knowledge	query, mode (keyword / semantic / hybrid), doc_types, filters, limit	\$search · \$vectorSearch · \$rankFusion	Finding anything by words or meaning
get_document	id, include (context, feature, runs)	find + \$lookup	Reading one doc in full, with its parent context
filter_test_cases	module, section, status, track, owner, environment	find()	Exact lists ("Marcelo's MySQL fails")
find_related	id, relation_types, depth (max 3)	\$graphLookup	"What is connected to this?"
find_by_entity	type, value	find on entities	Everything about patient 118 or ADT_A08
get_stats	group_by, filters	aggregate \$group	Counts and pass rates
trace_issue	start_id (TC id or Jira key)	several of the above	One-shot bundle: test + bug + code + context

MCP TOOL SCHEMA

What the LLM sees for

search_knowledge

```
{
  "name": "search_knowledge",
  "description": "Search test cases, features, context, Jira tickets and code. Use mode 'keyword' for IDs, codes and exact names; 'semantic' for questions in normal words; 'hybrid' when the question has both. Returns short Markdown snippets with ids for citation.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "query": { "type": "string" },
      "mode": { "type": "string", "enum": ["keyword", "semantic", "hybrid"], "default": "hybrid" },
      "doc_types": { "type": "array", "items": { "enum": ["test_case", "feature", "context", "jira_ticket", "code_chunk"] } },
      "filters": {
        "type": "object",
        "properties": {
          "module": { "type": "string" },
          "environment": { "type": "string" },
          "status": { "enum": ["PASS", "FAIL", "BLOCKED", "NA", "NOT_RUN"] },
          "track": { "enum": ["Firebird", "MySQL", "default"] }
        }
      },
      "limit": { "type": "integer", "default": 8, "maximum": 20 }
    },
    "required": ["query"]
  }
}
```

What a tool returns (small and easy to cite)

```
{
  "results": [
    { "id": "TC:INTEROP:eRx:R39", "doc_type": "test_case", "score": 0.91,
      "breadcrumb": "Interop > eRx > Testcase - 2 > RxFill",
      "title": "Send - RxFill request to OP - 2a",
    }
  ]
}
```

```

"snippet_md": "...Status is **Partially Dispensed** ... Reason Code is **Out of Stock**...",
"latest_status": { "track": "default", "status": "PASS" },
"cite": "InteropsTeamFunctionalityDetails > eRx > row 39" }
],
"next_hint": "Call get_document(id) for full steps, or find_related(id) for links."
}

```

Design rules for the tools

- **Few tools with clear names.** The descriptions tell the LLM *when* to use each one.
- **Never return vectors.** Return ids, breadcrumbs, short Markdown and a `cite` string.
- **Limit the size** (default 8 results, graph depth at most 3) so the LLM's context stays clean.
- **Enums for filters.** The LLM can't invent a status like "Passed!!".
- **Read-only.** The tools never write to the database.

8 How the LLM answers a question

Every question goes through the same five steps.

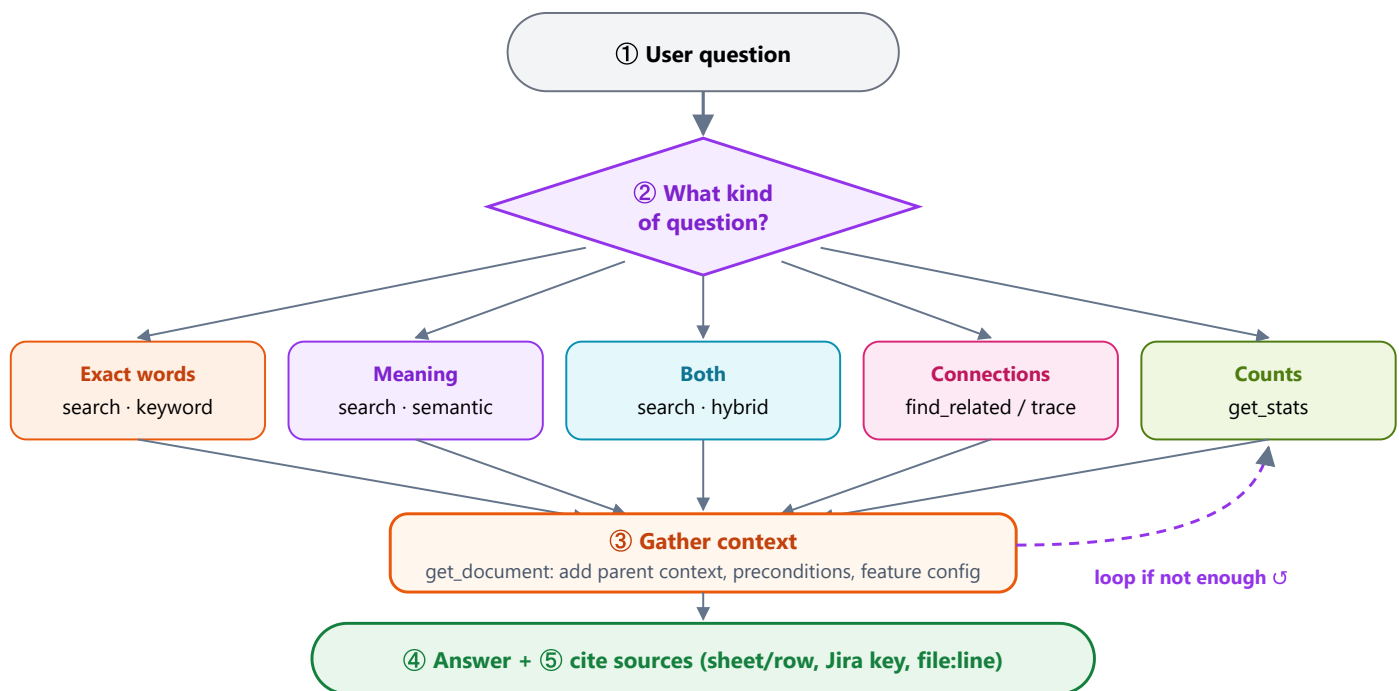


Figure 4 — The LLM can loop: if the first results are thin, it searches again or follows links.

9 Demos: how the LLM handles 5 question types

The test cases are real rows from your sheets. Jira and code are example data to show the flow.

Demo 1 · Keyword question

exact code

"Show me the steps for INT-RxFill-Naproxen-2a."

Sees a code → keyword →

search_knowledge(mode: keyword) →

get_document →

answer

```
{ $search: { index: "kw_index",  
  phrase: { query: "INT-RxFill-Naproxen-2a", path: ["search.keywords", "content_md"] } } }
```

Answer: This is the RxFill test in *eRx > Testcase - 2* (patient 123, Naproxen 500 mg). Steps: open Surescripts Message tester, respond to the NewRx as RxFill, choose template INT-RxFill-Naproxen-2a, confirm Quantity 15, Status *Partially Dispensed*, Reason *Out of Stock*, then Send. Expected result: a Success verification message. Last status: **PASS**.

Source: InteropsTeamFunctionalityDetails > eRx > row 39

Demo 2 · Semantic question

same meaning, different words

"Do we have a test for when the pharmacy can't give the patient the full amount?"

No codes, just an idea → semantic →

search_knowledge(mode: semantic, doc_types: test_case) →

answer

```
{ $vectorSearch: { index: "vector_index", path: "embedding.vector",  
  queryVector: embed("pharmacy can't give the full amount"),  
  filter: { doc_type: "test_case" }, numCandidates: 200, limit: 8 } }
```

Answer: Yes. "*Send - RxFill request to OP - 2a*" covers a **partial fill**: the pharmacy marks the Rx *Partially Dispensed* with reason *Out of Stock* (quantity 15 of 30). The next test (row 40) validates those fields in the XML.

Sources: eRx row 39, eRx row 40

 The question never says "RxFill", "partial" or "out of stock". The vector found it by **meaning**.

"Is there an ADT-A08 test for insurance updates that runs in environment 93?"

Code "ADT-A08" + idea "insurance update" + env →

search_knowledge(mode: hybrid, filters.environment: ID93) →

answer

```
{ $rankFusion: { input: { pipelines: {  
  keyword: [ { $search: { index: "kw_index", text: { query: "ADT-A08", path: "search.keywords" } } }, {  
    $limit: 20 } ],  
  semantic: [ { $vectorSearch: { index: "vector_index", path: "embedding.vector",  
    queryVector: embed("insurance update"), filter: { environments: "ID93" },  
    numCandidates: 200, limit: 20 } } ]  
} } } }
```

Answer: Yes: *HUB_ADT&SIU > TC 2.0 "Outbound_Verify when user update the patient demographics\Insurance..."*. Update a patient's insurance in OP, then check that Platform Manager > HUB > Messages receives an **ADT-A08** from the "Tanya ADT" third party. The ADT feature runs in **ID93, ID8 and ID10**.

Sources: HUB_ADT&SIU row 4 · IntropsFeatures row 14

"Why is the CancelRx response validation failing, and which code should a developer look at?"

filter_test_cases(module: eRx, status: FAIL, keyword: CancelRx) →

finds R27 →

find_related(R27, depth 2) →

gets R26, Jira, code →

get_document for each →

explain

```
db.qa_knowledge.aggregate([
  { $match: { _id: "TC:INTEROP:eRx:R27" } },
  { $graphLookup: { from: "qa_knowledge", startWith: "$relations.target",
    connectFromField: "relations.target", connectToField: "_id",
    as: "linked", maxDepth: 1, depthField: "hop" } },
  { $project: { "linked._id": 1, "linked.doc_type": 1, "linked.title": 1, "linked.summary": 1,
    "linked.runs": 1, "linked.status": 1, "linked.hop": 1 } }
])
// Jira tickets point TO the test (AFFECTS), so also run:
db.qa_knowledge.find({ "relations.target": "TC:INTEROP:eRx:R27", doc_type: "jira_ticket" })
```

Answer (the LLM joins the dots):

1. **The failing test:** eRx row 27 checks that the CancelResponse XML has Denial Reason, Code and Type. It is **FAIL**.
2. **The step before it:** row 26 sends that CancelResponse. It is marked Pass, but its comment says "*Cancel - Pending*", so the send may not have completed. ⚠️ The data looks inconsistent.
3. **Same check, other data:** row 57 (patient 123) has comment "*cancel pending*". That suggests the problem isn't tied to one patient.
4. **Known bug:** OPI-2045 (example), "Denial Reason Code missing", is In Progress.
5. **Where to look:** `CancelRxResponseParser.Parse`, `src/Surescripts/CancelRxResponseParser.cs` lines 42–118 (example).

Suggested next step: re-run R26 and confirm the CancelResponse really arrives, then re-test R27 after OPI-2045 is fixed.

Sources: eRx rows 26, 27, 57 · JIRA OPI-2045 · `CancelRxResponseParser.cs:42`

"Which module has the most MySQL failures right now?"

Counting → stats →

get_stats(group_by: module, track: MySQL, status: FAIL) →

answer with a small table

```
db.qa_knowledge.aggregate([
  { $match: { doc_type: "test_case" } },
  { $set: { latest: { $last: { $filter: { input: "$runs", cond: { $eq: ["$$this.track", "MySQL"] } } } } } },
  { $match: { "latest.status": "FAIL" } },
  { $group: { _id: "$hierarchy.module", fails: { $sum: 1 } } },
  { $sort: { fails: -1 } }
])
```

Answer: A ranked list such as "Billing 12, FrontDesk 7, Nurse 3...", with a link to see the failing tests in each module. (The numbers depend on live data. Your current sheets are mostly NOT_RUN.)

10 Why this helps the LLM find solutions and explain

Compare a plain "Excel rows as text" setup with this design, for the same question as Demo 4.

✗ Without this design

- The search finds row 27 alone. It doesn't know the patient or the drug, because that data was in row 2.
- It has no idea that row 26 must run first.
- It can't see the Jira bug or the code.
- "Pass" vs "pass " vs "Fail" can't be filtered reliably.
- The LLM **guesses**, and may make up a cause.

✓ With this design

- **context_snapshot** means every chunk knows its patient, drug and pharmacy.
- **DEPENDS_ON** means the LLM checks the step before.
- **AFFECTS / FIXED_BY** means the bug and the code are one hop away.
- **Clean status enums** mean exact filters and counts.
- **Citations** mean every claim points to a row, ticket or file. The LLM **explains** instead of guessing.

What the LLM can now do

Ability	Made possible by
Find the right test even with different words	Vector of <code>content_md</code> + <code>search.aliases</code>
Find exact codes and IDs every time	Keyword index on <code>search.keywords</code>
Know the full setup (patient, config)	<code>context_ref</code> + <code>get_document(include: context)</code>
Explain <i>why</i> something fails	<code>relations[]</code> + <code>\$graphLookup</code> + Jira + code
Do impact analysis ("what if this code changes?")	Walk links backwards: code → TESTED_BY → tests
Compare databases (Firebird vs MySQL)	<code>runs[]</code> with <code>track</code>
Spot bad data	<code>quality.flags</code> , e.g. Pass with a "Pending" comment
Be trusted	A <code>cite</code> on every result

11 Step-by-step rollout

Build it in small steps. Each step is useful on its own.

Step 1

Excel → MongoDB. Column mapping, sections, parent blocks, clean statuses, `runs[]`, `content_md`, embeddings. Create the 3 indexes.

Done when: all $\approx 2,770$ + interop tests are loaded, and a spot-check of 20 rows matches Excel.

Step 2

MCP server (read-only) with `search_knowledge`, `get_document`, `filter_test_cases` and `get_stats`.

Done when: Demos 1, 2, 3 and 5 work end to end.

Step 3

Links inside test data: `BELONGS_TO`, `USES_CONTEXT`, `DEPENDS_ON`, `SAME_SCENARIO`, plus the `find_related` and `find_by_entity` tools.

Done when: "what depends on R26?" returns R27.

Step 4

Jira: sync tickets updated since the last run, summarize comments, and create `AFFECTS` links.

Done when: each failed test shows its linked bug.

Step 5

Source code: function-level chunks, LLM summaries, `FIXED_BY` links from commit messages, plus the `trace_issue` tool.

Done when: Demo 4 works end to end with real data.

Step 6

Measure quality. Write 50 real questions with known right answers. Check that the right doc is in the top 5 and that every claim has a source. Tune keywords, aliases and weights.

12 Notes & limits

- **Dates:** the Excel sheets have no "tested on" column, so `executed_at` stays empty until testers add one. `recorded_at` is set by the import.
- **Jira and code examples** in this document (OPI-2045, `CancelRxResponseParser`) are made up to show the flow. Replace them with real ones.
- **MongoDB version:** `$vectorSearch` and `$search` run on Atlas, or self-hosted with the search process (`mongot`). `$rankFusion` needs 8.1+. Check your exact version first.
- **Embedding model:** pick one and keep it. Changing models means re-embedding everything. Store the model name in `embedding.model`.
- **Security:** the test data has patient names. Keep the MCP server read-only, behind login, and restrict who can query it.